ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY

LAB TUTORIAL

Lab Tutorial 2.1 – Compiling with GCC

Target	Related theory	Resources
Hello Word	Ch1: Command line interface	Use Ubuntu 16/18 image
Compile with gcc	Ch2.5 Linker & Loader	Use Windows 10 Use Cocalc

Ask students: Understand the theory involved. Know how to compile sample code and execute it. Know how to observe processes inside Windows and Ubuntu systems. Apply the commands introduced to complete the exercise.

Student reviews: Questions and answers about theoretical issues. Practical skills. Exercise.

Submission requirements:.c source files and .out executable files for the "examples" and exercises at the end of the tutorial. Screenshot of progress information.

Preferences

[1] Chua Hock-Chuan, GCC and Make - Compiling, Linking and Building C/C++ Applications, Nangyan Technical University, Singapore, March 2018

[2] Abraham Silberschatz, Peter B. Galvin, Greg Gagne, [2018], Operating System Concepts, 10th edition, John Wiley & Sons, New Jersey.

Chapter 2.5 – Linker and Loader.

1. Install and test GCC

The GNU Toolchain, which includes GCC, is included in Ubuntu as the standard compiler.

To check GCC version, type:

```
$ gcc --version
gcc (GCC) 6.4.0
```

Some additional information can be found using the -v option

```
$ gcc -v
```

Or seek help using the -help option

```
$ gcc --help
```

To read GCC's instructions, type:

```
$ man gcc
// Press space key for next page, or 'q' to quit.
```

To export instructions to a text file, type:

```
$ man gcc | col -b > gcc.txt
```

Additionally, guidance can be found herehttp://linux.die.net/man/1/gcc.

Or search in the folder "usr/share/man/man1".

```
$whereis gcc
gcc: /usr/bin/gcc.exe /usr/lib/gcc /usr/share/man/man1/gcc.1.gz
```

2. Compile the first program

The GNU C compiler is gcc and for C++ it is g++.

Create a source file hello.c and enter the following content.

```
first // hello.c
2     #include <stdio.h>
3     int main() {
4     printf("Hello, world!\n");
5     return 0;
6     }
```

To compile the newly created hello.c file, use the command:

```
> gcc hello.c
```

The default output program name is "a.out" (Unixes and Mac OS X).

To run the program you just created, type:

```
$ ./a.out
```

*In some situations, prior authorization is required

```
$ chmod a+x a.out
$ ./a.out
```

Notes for Unix and Bash Shell:

- In the Bash shell, the default PATH does not include the current working directory. Therefore, you need to include the current path (./) in the command.
- The full program name should be typed including the file extension, if any, i.e., "./a.out".
- In Unix, the output file can be "a.out" or simply "a". Furthermore, you need to assign the executable file mode (x) to "a.out", via the command "chmod a + x filename" (add executable file mode "+ x" to all also user "a + x").

To specify the output filename, use the -o option:

```
$ gcc -o hello hello.c
$ ./hello
```

Notes for Unix

- In Unix, we often omit the .exe file extension (Windows only) and simply name the executable file hello.out, or without the extension, hello.
- It is necessary to assign the executable file mode via the command "chmod a + x hello".
- We use g++ to compile C++ programs (rare in this course).

Some options with GCC

Commonly used options include:

```
$ gcc -Wall -g -o Hello.out Hello.c
```

- -o: specifies the output program file name.
- -Wall: prints all Warning messages.
- -g: generate additional symbolic debugging information for use with the gdb debugger.

Compile and link separately

The above command compiles the source code file into an output program through a single step. We can separate it into two stages: compile the source code file(s) into object file(s) and link the object file(s) with system libraries to produce program files. final process with the following options:

```
// Step 1 with -c option
> gcc -c -Wall -g Hello.cpp
// Step 2 with -o option
> gcc -g -o Hello.exe Hello.o
```

The options are:

- -c: Compile to object file "Hello.o". By default, the object file has the same name as the source file and has the extension ".o" (no need to specify the -o option). It is not linked to other object files, if any, and has nothing to do with system libraries.
- Linking is done with input ".o" (rather than ".c") object files. GCC uses a separate linking scheme to perform linking.

Exercise

1. Check gcc version on your machine.
2. Compile hello.c to hello.out and execute. Submit file hello.out
3. Edit the file myprogram.c which allows entering from the keyboard a person's name (see as string X) and printing "Hello X". Compile to myprogram.out and execute.
4. Edit the loop.c file which contains the endless loop. Compile to loop.out and execute. Meanwhile, switch to the "System Monitor" utility and observe what ID and name the currently executing program has and how it occupies CPU and RAM?
5. (Optional) Exchange .out files between students and execute this file on other students' machines.
6. (Optional) Using cygwin in the Windows environment to compile the hello.c file, how do we get the executable file?